Assignment 2:

Simulating a Simple Cache Controller FSM

Name of the students:

AKM Azimul Ashique Khan

ID: 230041153

Mahir Labib

ID: 230041139

Submission date: 16-April-2026

Github Repo Link: https://github.com/Ashique-7/Simple-Cache-Controller-Simulator

Introduction:

Cache memory is used in order to reduce the average memory access time by storing the data that are frequently accessed closer to the memory. However, this cache needs to be managed. In order to manage the cache, various cache operations are used like hits, misses and memory access. These operations need a control or management mechanism.

In the project we developed, we used a FSM (Finite state machine) to implement a simple cache controller. The FSM can control how the cache react to the read or write requests, can handle hits and misses and also manage the communication with the main memory.

The Simulator was implemented in C++ and the simulator works such that the FSM operated cycle by cycle and it can also show how the cache transitions between the states.

System Design:

The system comprises of 3 main elements:

1. ***CPU (Request Generator):*** The CPU provides the read and write requests. Each request contains the type(read or write), a 32 bit address and for write operations exclusively, write data.
2. ***Cache:*** It has a direct mapped structure. It consists of 1024 cache lines. Each line contains valid bit, dirty bit, tag and 4 words. Here 4 words is the block size.
3. ***Main Memory:*** The main memory is simulated using a map. The main memory responds after a certain amount of delay. In my particular project, the memory latency is equal to 4 cycles. The main memory supports both reading a block or writing on a block.

Cache specifications:

In my project, the cache was designed according to the specifications provided in the textbook. That implies:

- Cache type: Direct-mapped
- Write policy: Write-back
- Allocation policy: Write-allocate
- Block size: 4 words (128 bits)
- Total lines: 1024

In the request generator we saw that a 32 bits address was passed. The breakdown of that address is as follows:

Field	Bits	Description
Tag	[31–14]	Identifies memory block
Index	[13–4]	Selects cache line
Word Offset	[3–2]	Selects word in block
Byte Offset	[1–0]	Selects byte

Finite state machine design:

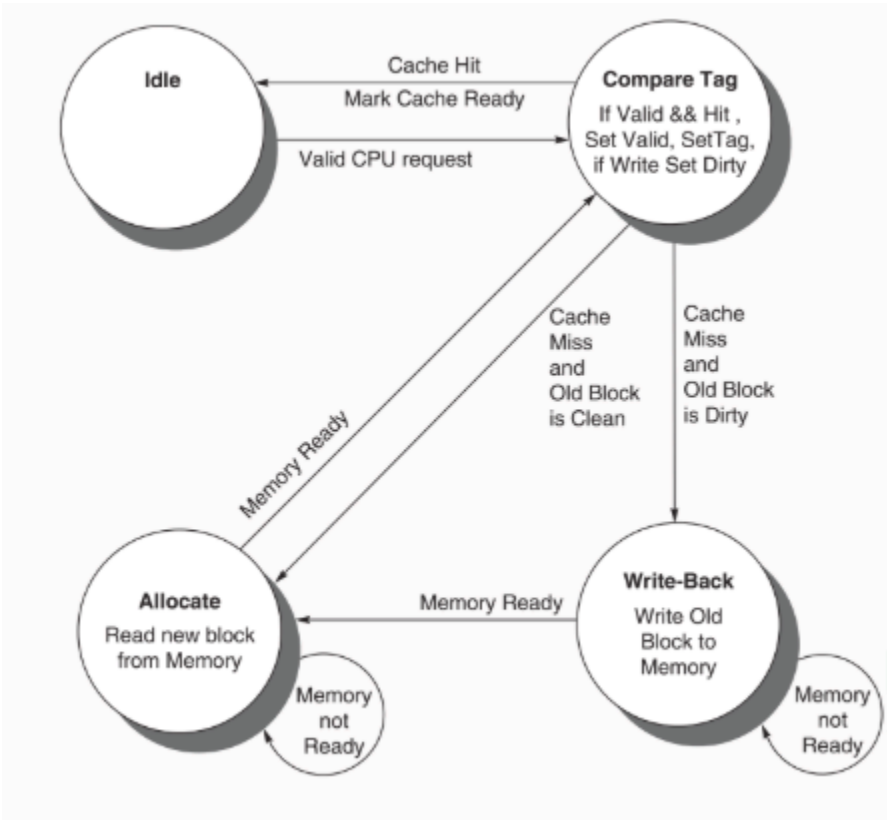
The controller is implemented as a moore device. So, the output depends only on the current state.

The states are :

- IDLE STATE: During this state, the cache waits for a cpu request. When the request arrives the state transitions to Compare Tag state.
- Compare Tag: During this state, the FSM extracts index and tag from the address. The compare tag state also checks the valid and dirty bits.
  If HIT:
  Read → return data
  Write → update data and set dirty bit
  Transition → IDLE
  If MISS:
  If dirty → Write back state
  Else → Allocate state
- Write back: Writes the dirty block to memory. After writing a block, it waits for memory latency cycles. After completing write back , the state transitions to Allocate state.
- Allocate: In this state, the controller fetches block from the memory and updates cache line:
  valid=1;
  dirty=0;
  tag also updated.

  then the state transitions to the Compare tag state.

The diagram for the transition of states is as follows:



Implementation:

FSM Implementation:

The finite state machine was implemented using an **enum State.** It was controlled using a switch-case structure and the state transitions occurred on every simulation cycle.

```
enum class State : uint8_t {
    IDLE,
    COMPARE_TAG,
    WRITE_BACK,
    ALLOCATE
};
```

The cache was implemented using structure declaration:

```
struct CacheLine {
    bool valid = false;
    bool dirty = false;
    uint32_t tag = 0;
    uint32_t data[BLOCK_WORDS] = {};
};
```

The CPU request was also declared as a struct and contained a bool indicating where the request was read or write, an address, a write data and a label.

The address was decoded into index and tag by right shifting it by 4 bits and 14 bits respectively. We had to AND the value of right shift 4 with 0x3FF because the binary of 0x3FF is 00000011 11111111 where 10 bits are set to 1. So I can obtain the 10 bits from the result.

Memory Simulation: The memory was simulated using a map. It simulated the block transfers and was controlled using a countdown timer names memTimer. The main memory had 2 functions; one to read the block and the other to write the block.

The cache controller divides each of the memory address into index and tag using the method mentioned earlier and verifies whether the data requested by the CPU is present or not. In the run loop, the controller stays in IDLE STATE. After fetching a request, it moves to compare tag state where it checks for the cache hit or miss. If it hits, it reads or writes the data based on the request. If the data is written the cache line is marked dirty. If the cache misses, it checks where the line is dirty or not. If the existing cache line is both valid and dirty, then it decides to write back first then allocate. Other wise, It directly allocates. During write back, the dirty data is written on the main memory and during allocate, a new block is fetched to the cache and the meta data like tag, valid bit or dirty bits are updated accordingly. In this way all the requests are handled. Also memTimer=memory_latency here indicates that this memory operation will take more time. So when memTimer hits 0, it indicates that the memory operation is completed.

Main function: A number of different Test cases were taken and a cache object was created. Then the test cases were run on the cache. we obtain the result in a different txt file.

Simulation:

Here, 4 test cases were taken.

1. TC1: Read miss ---> Cache is empty so block is fetched from the memory.

FSM sim:

COMPARE TAG → ALLOCATE → COMPARE TAG → IDLE

2. TC2: Read hit ---> Data is found in cache. FSM sim: COMPARE TAG → IDLE
3. TC3: Write hit ---> Data updated in the cache so dirty bit set to 1.
   FSM sim:
   COMPARE TAG → IDLE
4. TC4: Dirty miss --->Block in cache is dirty, so write back is required before the allocation.
   FSM sim:
   COMPARE TAG → WRITE BACK → ALLOCATE → COMPARE TAG → IDLE

Conclusion:

This project successfully demonstrates how a finite state machine controls cache system. The implementation follows the specifications from the textbook and clearly shows how different scenarios like the cache hit or miss or even dirty miss affect the state transitions. It has its fair share of limitations too. Like there is no pipelining or parallel memory operations. Also the memory has a fixed latency. So there are some possible future improvements. They are:

1. Since the cache is blocking cache i.e. CPU must wait until one cache operation finishes before starting the next one, we need to implement a non-blocking cache.
2. A write buffer can be implemented.
3. The compare and access stages can be separated for the design to be more understandable.